

Session 11: PointersTopics:Pointer declaration, dereferencing, pointer arithmetic.Example: Swap numbers using pointers.Trainer Demo: Use memory diagram to explain address referencing.

Tasks

1.

Declare an integer variable called likes and a pointer variable called ptrLikes; assign likes a value, point ptrLikes to likes, and print both the value and the address stored in ptrLikes.

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int likes = 5000;
```

```
    int *ptrLikes;
```

```
    ptrLikes = &likes;
```

```
    printf("Value of likes = %d\n", likes);
```

```
    printf("Address of likes = %p\n", (void *)&likes);
```

```
    printf("Address stored in ptrLikes = %p\n", (void *)ptrLikes);
```

```
    printf("Value using ptrLikes = %d\n", *ptrLikes);
```

```
    return 0;
```

```
}
```

```
C:\Users\gajja\OneDrive\Desktop >
Value of likes = 5000
Address of likes = 000000000062FE44
Address stored in ptrLikes = 000000000062FE44
Value using ptrLikes = 5000

-----
Process exited after 0.1013 seconds with return value 0
Press any key to continue . . .
```

2.

Write a function `swapPlaylistCounts(int *a, int *b)` that swaps the number of songs in two Spotify playlists using pointers, then call the function in main and print the swapped values.

```
#include <stdio.h>
```

```
void swapPlaylistCounts(int *a, int *b)
```

```
{
```

```
    int temp;
```

```
    temp = *a;
```

```
    *a = *b;
```

```
    *b = temp;
```

```
}
```

```
int main()
```

```
{
```

```
    int playlist1 = 50;
```

```
    int playlist2 = 80;
```

```
    printf("Before swapping:\n");
```

```
    printf("Playlist 1 = %d songs\n", playlist1);
```

```

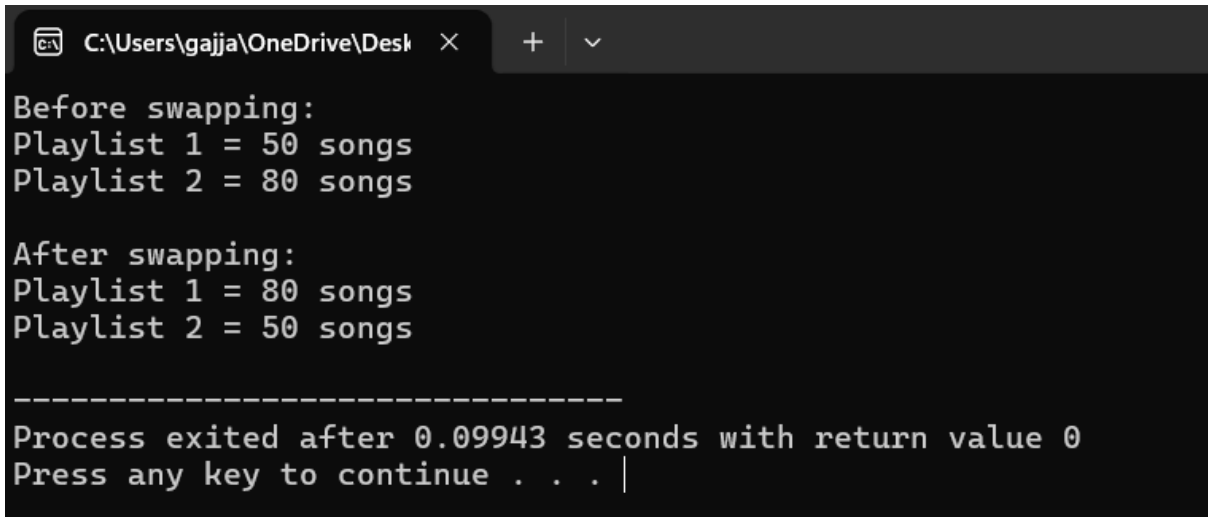
printf("Playlist 2 = %d songs\n", playlist2);

swapPlaylistCounts(&playlist1, &playlist2);

printf("\nAfter swapping:\n");
printf("Playlist 1 = %d songs\n", playlist1);
printf("Playlist 2 = %d songs\n", playlist2);

return 0;
}

```



```

C:\Users\gajja\OneDrive\Desktop
Before swapping:
Playlist 1 = 50 songs
Playlist 2 = 80 songs

After swapping:
Playlist 1 = 80 songs
Playlist 2 = 50 songs

-----
Process exited after 0.09943 seconds with return value 0
Press any key to continue . . . |

```

3.

Given an array of 5 order amounts (e.g., Zomato orders), use a pointer to iterate through the array and print each amount along with its memory address.

Hint: Use pointer arithmetic to move to the next element.

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int orders[5] = {250, 450, 180, 320, 500};
```

```

int *ptr = orders;

for(int i = 0; i < 5; i++)
{
    printf("Order amount = Rs.%d\tAddress = %p\n",
        *(ptr + i), (void *) (ptr + i));
}

return 0;
}

```

```

C:\Users\gajja\OneDrive\Desktop
Order amount = Rs.250    Address = 000000000062FE20
Order amount = Rs.450    Address = 000000000062FE24
Order amount = Rs.180    Address = 000000000062FE28
Order amount = Rs.320    Address = 000000000062FE2C
Order amount = Rs.500    Address = 000000000062FE30
-----
Process exited after 0.1079 seconds with return value 0
Press any key to continue . . . |

```

4.

Create a function `incrementFollowers(int *followers, int n)` that increases each follower count in an array (representing Instagram followers for 5 friends) by 100 using pointer arithmetic, then print the updated counts.

```
#include <stdio.h>
```

```

void incrementFollowers(int *followers, int n)
{
    for(int i = 0; i < n; i++)
    {
        *(followers + i) = *(followers + i) + 100;
    }
}

```

```

}

int main()
{
    int followers[5] = {1200, 2500, 3400, 4100, 5000};

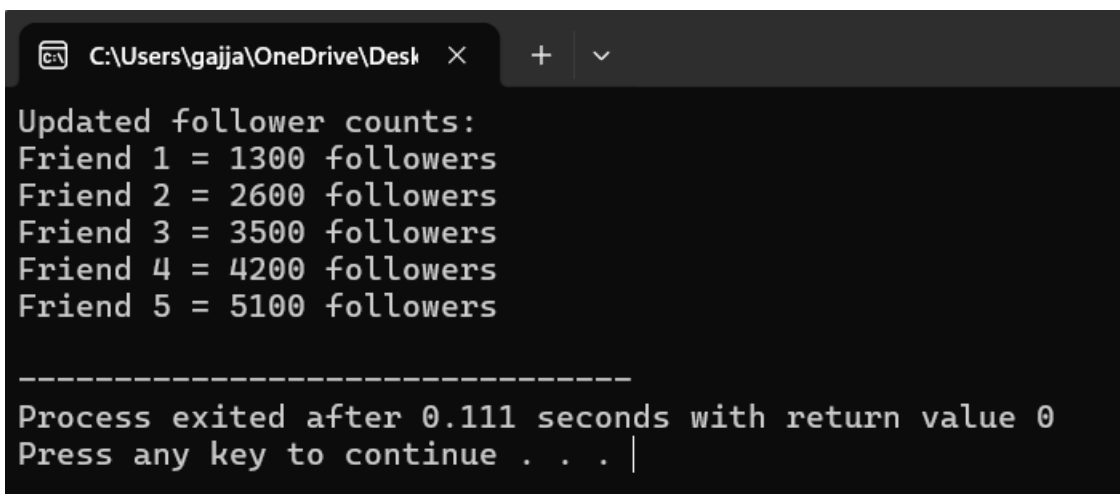
    incrementFollowers(followers, 5);

    printf("Updated follower counts:\n");

    for(int i = 0; i < 5; i++)
    {
        printf("Friend %d = %d followers\n",
            i + 1, followers[i]);
    }

    return 0;
}

```



```

C:\Users\gajja\OneDrive\Desktop >
Updated follower counts:
Friend 1 = 1300 followers
Friend 2 = 2600 followers
Friend 3 = 3500 followers
Friend 4 = 4200 followers
Friend 5 = 5100 followers

-----
Process exited after 0.111 seconds with return value 0
Press any key to continue . . . |

```